

Paraphrase Detection in a Low Resourced Language: Kannada

Anagha H M

Department of Computer Science
PES University
Bangalore, Karnataka
anu.hm0520@gmail.com

Janya Mahesh

Department of Computer Science
PES University
Bangalore, Karnataka
janyamahesh@gmail.com

Karthik Sairam

Department of Computer Science
PES University
Bangalore, Karnataka
karthik.sairam2001@gmail.com

Dr. Mamatha HR

Department of Computer Science
PES University
Bangalore, Karnataka
mamathahr@pes.edu

Abstract—Paraphrase detection is a Natural Language Processing classification problem whose main goal is to determine if two sentences are synonymous. The research undertaken in this paper strives to implement the same in Kannada, a low resourced language. Kannada is a South-Indian language that originates from the Dravidian language. On comparison of many appropriate state-of-the-art classifiers for this problem, the decision tree gave the best results due to the non-linear aspect of the corpora. Such a system would be best suited to implement a plagiarism detection system for Kannada documents that are in the official dialect of the Kannada language.

Index Terms—Natural Language Processing(NLP), tokenization, stemming, decision trees, classifier, indicNLTk(iNLTk)

I. INTRODUCTION

Text processing has always been an extremely important aspect of natural language processing. Much research has been carried out in this domain pertaining to English and many European languages, leaving behind many low-resourced languages. One such language is Kannada, a Dravidian language. Kannada is a language used for conversation and administrative purposes by the people of Karnataka, a southern state in the subcontinent of India. Working towards developing a paraphrase detection system in Kannada requires the building of relevant corpora, NLP tools like the stemmer and tokenizer. The related learning models built from scratch acts as the basis of further developing a fully working robust model for paraphrase detection.

To determine whether the two sentences are synonyms, paraphrase detection is utilised. The task that needs to be addressed first involves the building of relevant dataset. Keeping the required corpora in mind, the dataset have to be built so as to primarily resource the project. In addition to the originally crafted dataset, the MSRP corpus dataset [1] [2] was also used in conjunction so as to increase the data that the model is trained on.

The next task is the preprocessing of the built dataset. Preprocessing of the data includes removal of unnecessary

symbols, numbers from the dataset. This ensures that the dataset is ready for the next step of processing, which is tokenization. Tokenization is a fundamental process in NLP. This involves obtaining the visually smallest units of a sentence, the tokens. Tokens are not necessarily the words in the sentence, but the smallest meaningful unit of the sentence. They can be words, subwords, or even characters. Subwords refer to a group of letters that don't exactly have a meaning. Character tokens refer to a single alphabet. Once the sentences are tokenized, they need to be stemmed to extract the root words.

After all the above-mentioned steps are completed, the similarity metrics are computed and are forwarded to multiple classifiers. These classifiers are trained using the similarity metrics input to them, and is tested with relevant testing tools and techniques to achieve maximum accuracy possible.

Building a complete paraphrase detection system in the Kannada language not only increases the research done in this field, but also necessitates further research that would allow the horizon for research in Kannada NLP to expand. Due to the lack of such basic NLP tools for Kannada at the present moment, the need has never been more. Such a system would set a benchmark for paraphrase-related tasks in the future. In the following sections, the literature review done to learn about the previous work done pertaining to paraphrasing, and the methodology and architecture undertaken to build such a system are detailed. In addition, computation of accuracies of the learning models and optimum variables are also presented.

II. LITERATURE SURVEY

Much work has been conducted in paraphrase detection primarily in English and other sought after languages.

The authors of [3] focused on building a paraphrase generator that uses the n-gram approach. The n-gram outputs the main phrases in the given document. This approach was implemented for the Kannada language. The authors use an

algorithm to match the n-grams found with the reference paraphrases. “The R-Precision metric received an accuracy of 46.3% for a corpus containing 1000 sentences.” Since the relevance of the algorithm is limited to monolingual corpus’ input, this can be identified as a drawback of the proposed model. n-gram approach requires a granular understanding of natural language processing techniques to implement in Kannada. Also, Paraphrase generation is a slightly easier approach than paraphrase detection.

The authors of [4] focus on the compound words, which is an amalgamation of two standalone words, or root words. These compound words stem from the semantic grammar of the Sanskrit language and hence, all the languages that originate from the Sanskrit script are applicable to semantically process the compounds. The methodology includes segmentation, deciding a structure, relations identification and paraphrase generation. Segmentation involves splitting the compound word into two words, and in Sanskrit languages the compound word is written without any space, unlike in English where a compound word is represented with a hyphen or a space in between. The constituency structure outputs a binary tree of the syntactic structure. The type identifier is used to identify the type of compound like “Endocentric, Karmadharaya, Exocentric, Copulative Compounds”. Some types of compounds are more common than the others, this is not taken into account when creating the architecture. Additionally, Hindi data presents many cases where quite a lot of compounds require a verb as well as an extra word for its paraphrasing, which are not handled.

The authors of [5] propose a methodology “used a traditional logistic regression classifier” for implementing the system that allows the identification of paraphrases in Indic languages. The various similarity measures used are as follows:

- Cosine Similarity
- N-gram Based Similarity
- Word Overlap- Exact Match
- Stemmed Word Overlap
- Semantic Similarity

The performance of the system is measured as an F1-score of 0.81.

Lemmatizer and stemmer are two very important tools in NLP. The authors of [6] propose an approach to create a rule-based lemmatizer. Lemmatizers remove affixes by applying linguistic rules. This paper proposes the rules to identify affixes and design a lemmatizer in Kannada. The algorithm checks if a given word has a prefix or a suffix, if it does not, the given word is the lemma. If it does, depending on whether the word has a prefix or suffix, those are respectively removed from the word to get the lemma. The prefixes and suffixes are identified by the rules given. The accuracy of the given rule-based lemmatizer is 85%. This can be improved by identifying more prefixes and suffixes. It can also be improved by adding better rules.

The paper [7] proposes a method to perform suffix stripping on a Kannada corpus. This is a crucial part of information

retrieval in Kannada. IR tools in languages are not very popular and hence it is a challenge to develop them in regional languages. In Kannada, Suffixes are called Pratyayas. Suffixes are of different types too, such as suffixes indicating plurals, suffixes defining the object type, suffixes defining gauge/quantity, suffixes with adjectives, and suffixed for verbs. The approach used here is an iterative method. All the suffixes are repeatedly removed until only the root word is remaining. This is done since a word can have multiple suffixes. After stripping off the suffixes, by adding a vowel at the end, the generation of the root word is completed. This is done since most Kannada words end with a vowel. The relevant documents from the Kannada magazine ‘Kendasampige’ serves as the dataset here. This approach gives an F1 score of 68.57%.

The authors of [8] propose a model to build a corpus in Kannada. This is done using a suite of PERL programs. From the research done, it is found that there are very few existing Kannada corpora. This paper proposes a solution to this issue. Some of the sources for Kannada works and texts used to create the dataset are Wikipedia, Prajavani (a Kannada newspaper), Sampada, and web blogs. This process starts by selecting a set of seed words. This corpora has roughly a million words. The middle frequency words are chosen from this dataset. Given these words, the algorithm constructs queries. Seed URLs are extracted from these queries, from the search engines. The web is then crawled using these URLs. The information retrieved from web crawling is filtered and preprocessed. It is filtered by size, function words. The HTML links are removed, near-duplicates are detected and removed, and language encoding is detected. After the preprocessing, tokenization, lemmatization, and POS Tagging are done. The tool is built using PERL under Linux. The corpus built using this algorithm has words, and the corpus needed for our problem statement is sentences. Thus, this has to be extended into extracting sentences.

[9] proposes a model trained on word embeddings for Kannada. This paper demonstrates word embedding on Kannada sentences to encode the semantic properties of words. The model is evaluated on tasks of text classification and similarity tasks. Word embedding is the basic for advancing in many popular domains in NLP. With the lack of dataset in low resource languages, it is very easy to overfit the model. This paper uses the monolingual corpora provided by IndicNLP. After cleaning the data, which includes removing foreign tokens, large sentences and duplicates, the corpora has 11 million sentences. Tokenization is done either using SentencePiece or BertWordPiece. The authors used 4 models for this task:

- Word2Vec
- FastText
- RoBERTa
- ELECTRA

For smaller vocabulary, the Word2Vec and FastText almost provided same results of text classification, but differed greatly with larger vocabulary. For smaller vocabulary, the Word2Vec

and FastText almost provided the same results of text classification, but differed greatly with larger vocabulary. The word similarity results for Word2Vec contain pronouns as it infers word as a token value. The drawback of this paper is that it is domain specific.

The authors of [10] discusses various paraphrase identification methods, comparing the observations implemented on Indian languages. Textual Similarity metrics uses various distance metrics. The SumoMetric outperformed other measures. Weighted dependency and word semantics for computing degree of similarity and dissimilarity. The ratio of the similarity score to the dissimilarity score was used to calculate the paraphrase score. This paper discusses various similarity metrics like Identical Phrase and Similar Word Matching , String similarity with synonyms , Semantic relatedness, Statistical and semantic analysis, UNL graph matching method, Bi-CNN-MI. Semantic relatedness outperforms other methods. Some of the techniques used in this paper enlighten the broad spectra of research in the area of NLP for paraphrase detection.

III. PROPOSED METHODOLOGY

The proposed methodology involves the following steps - Creation and cleaning of the dataset, pre-processing, tokenization, stemming, similarity metric computation, training and testing classifiers. The methodology follows the architecture model given in Fig. 1. This allows the conceptual problem statement to be implemented to show the operation of the proposed algorithm.

A. Creation of the Dataset

The first step includes the dataset in the Kannada language to be worked on. This is indeed challenging due to the lack of meaningful datasets in the Kannada language, and hence, a new dataset was created from the ground up.

The dataset thus created has over 600 data points and over 1200 sentences in the Kannada language. The data points are primarily in the Kannada language, but also sparsely contain English Nouns like country names (e.g, U.S.A) and numerical data (e.g, 6%). Certain alphanumeric characters also exist, like the percentage symbol (%) and the dollar sign(\$). The Quality column in Table I contains binary values, a value of '1' if the corresponding 'Sentence 1' and 'Sentence 2' cells are paraphrases of each other, or a value of '0' if they are not. The 'Sentence 1' and 'Sentence 2' columns contain one Kannada sentence each that might or might not be paraphrases of each other.

TABLE I
STRUCTURE OF THE DATASET

Sentence 1	Sentence 2	Quality
..	..	..

B. Dataset Acquisition

In addition to the created dataset, the corpus provided by the MSRRC [1] [2] was used. It contains 5800 sentence

pairs in English, including the 'Quality' column mentioned in Table I. The corpus is divided into the training and testing datasets by the authors themselves. Since the sentences are in English, these sentences have to be translated in the Kannada language, which was done using the Google Translator API. The translated sentences were validated by the authors of this paper, and the irrelevant and illogical sentences were pruned. Few data points of the complete dataset are shown in Fig. 2.

Sentence_1	Sentence_2	Quality
ಪಿಸಿಡಬ್ಲ್ಯೂನ ಮುಖ್ಯ ಕಾರ್ಯನಿಷ್ಠಾಪಕ ಮುಖ್ಯ ಕಾರ್ಯನಿರ್ವಹಣಾ	ಬೇಸಿಗೆಯ ಕೊನೆಯಲ್ಲಿ ಮಾರಾಟದ ಮತ್ತು ನಂ ಪೂರ್ವ ಮೋಟಾರ್ ಕ	1
ಫೆಡರಲ್ ಸೆಂಟರ್ಸ್ ಫಾರ್ ಡಿಸೀಸ್ ; ರಲ್ಲಿ ಯುನೈಟೆಡ್ ಸ್ಟೇಟ್ಸ್ ದಡಾ	ಗಲ್ಪ ಆಫ್ ಮಕ್ಸಿಕೋದಲ್ಲಿ ಉಷ್ಣವೇ ಭಾನುವಾರ ಗಲ್ಪ ಆಫ್ ಮಕ್ಸಿಕೋದ	1
ಕಂಪನಿಯ ಬದಲಿ ಮತ್ತು ರಿಪೇರಿ ಎ ಆದರೆ ಕಂಪನಿಯ ಅಧಿಕಾರಿಗಳು ಬ	ಇತ್ಯರ್ಥಪಡಿಸುವ ಕಂಪನಿಗಳು ಹೊ ಒಪ್ಪಂದದ ಅಡಿಯಲ್ಲಿ ಇತ್ಯರ್ಥಪಡಿ	0
ಏರ್ ಕಮೋಡೋರ್ ಕ್ಲೈಫ್ ಅವರು ಭದ್ರತಾ ಕಾರ್ಯಾಚರಣೆಯು ಅಭ್ಯಾ	ವಾಷಿಂಗ್ಟನ್ ಕೌಂಟಿಯ ವ್ಯಕ್ತಿಯೊಬ್ಬ ಈ ವರ್ಷ ವೆಸ್ಟ್ ನೈಲ್ಸ್ ಕೌಂಟಿಗಳ	0
ಮೊಸ್ಕಿ ಮತ್ತು ಹಿರಿಯ ಸಹಾಯಕರ ನವಾಡಾದ ನಲ್ಲಿಸ್ ಏರ್ ಪೂರ್ವಾ	ವಿಶಾಲವಾದ ಸ್ವಾಂಂಡರ್ಡ್ & ಪೂವ ಟೆಕ್ನಾಲಜೀಸ್ ನಾಸ್ಕಾಕ್ ಕಾಂಪೋ	1
		0

Fig. 2. Dataset created

C. Cleaning of the Dataset

Dataset cleaning involves the removal of unwanted characters and numbers from the data points in the dataset. Given below is the list of characters classified as 'unwanted'.

- English Alphabets
- Special Characters (\$, %, &, ., -, ,, ", ' ,)
- Numerals (0-9)

The dataset is then checked for inconsistencies, or NULL values. The final step in completion of dataset cleaning is to export the dataset thus cleaned into a suitable format without the loss of any data points, or part of any data point. This was done by exporting the file into a CSV (Comma-Separated Values) file.

D. Tokenization

Any text processing task begins with splitting the sentences into their words, namely tokens. Tokenization is more than just separating the words; it involves finding the smallest meaningful unit in a sentence. Such a unit is called a token. The tokenization process is implemented using the INLTK library, which is a toolkit for natural language processing in Indic Languages.

E. Stemming

The tokens retrieved after tokenization might not be in the root word form. Hence these tokens need to be further processed to extract the root word(lemma). This process is called as stemming and the tool used to perform this is known as the stemmer. The stemmer has a set of rules to identify the affixes (includes both prefixes and suffixes) and it takes the tokens from the tokenizer as it's input and removes all the suffixes and prefixes. Suffix is one or more letters added to the end of the word and a prefix is one or more letters added to the beginning of the word. The absence of a Kannada stemmer inspired us to build one. This required us to establish the rules

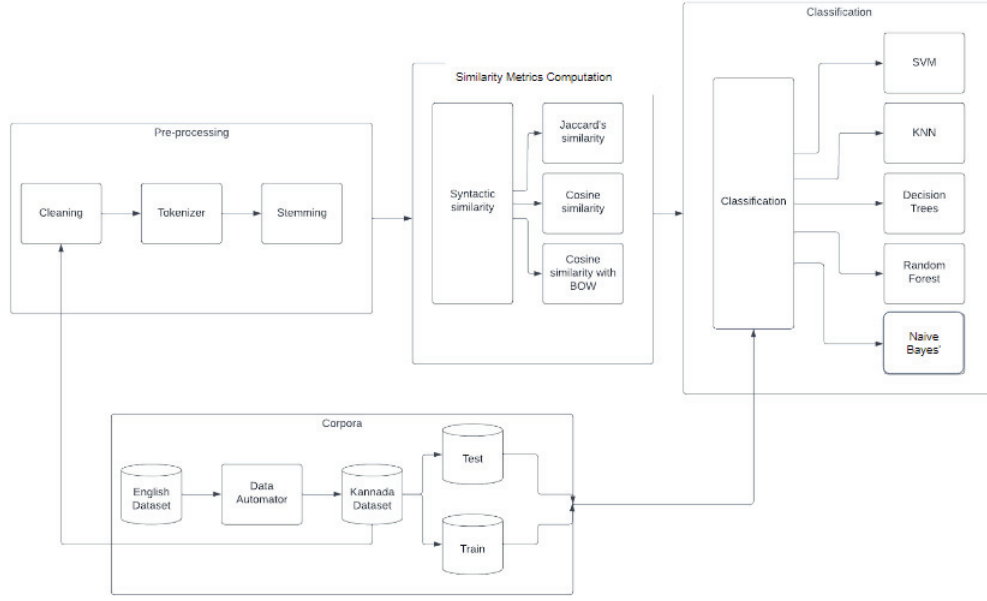


Fig. 1. Architecture diagram

to identify affixes. A thorough study of the Kannada language was essential to define the rules. A total of 73 rules were defined, some of which are shown in Fig. 6.

- 1 [,"ವಾಗಿದ್ದರು", "ಯಾಗಿದ್ದರು", "ದಾಗಿದ್ದರು", "ತಾಗಿದ್ದರು", "ದಿದ್ದರು"]
- 2 [,"ದ್ದನ", "ದಿದ್ದ", "ರಲಿಲ್ಲ"]
- 3 [,"ಗಾದ", "ಯಾದ", "ರಾದ", "ವಾದ"]
- 4 [,"ಗಾಣುವ", "ಗಾಗಿದ", "ಗಾಣುತ್ತಿದ್ದ"]]

Fig. 3. Stemmer Rules

A token can have multiple affixes. Hence the stemming process is repeated until there are no more affixes. This leads to the root word.

F. Similarity metric computation

Similarity metric computation refers to the depiction of how similar the 2 data points are in terms of numerical values. This should be done while ensuring that there is little to no loss of the original data. Similarity metric computation reduces dimensionality. Some of the techniques used to perform similarity metric computation are as follows:

- Cosine similarity : It is used to measure the text-similarity irrespective of the size.
- Jaccard's similarity : It is used to measure which members are distinct and shared.
- Cosine Similarity with Bag of Words : The text is treated as a bag of words, ignoring grammar, multiplicity.

The similarity metrics extracted are added to the dataset as shown in Fig. 4. These similarity metrics are then passed on to a classifier which uses them to make predictions.

Sentence_1	Sentence_2	Quality	COSINE(BOW)	Cosine Similarity	Jaccards
ಪ್ರಸಿದ್ಧವಿರುವ ಮುಖ್ಯ ಕಾರ್ಯನಿರ್ವಹಣಾ ಮುಖ್ಯ ಕಾರ್ಯನಿರ್ವಹಣಾ	ಪ್ರಸಿದ್ಧವಿರುವ ಮುಖ್ಯ ಕಾರ್ಯನಿರ್ವಹಣಾ ಮುಖ್ಯ ಕಾರ್ಯನಿರ್ವಹಣಾ	1	0.8033264177	0.7732812166	0.5925925926
ಭವಿಷ್ಯದ ಕೊನೆಯಲ್ಲಿ ಮಾರಾಟದ ಮತ್ತು ನಂ ಪೋರ್ಟ್ ಮೊಟೋ ಕ	ಭವಿಷ್ಯದ ಕೊನೆಯಲ್ಲಿ ಮಾರಾಟದ ಮತ್ತು ನಂ ಪೋರ್ಟ್ ಮೊಟೋ ಕ	1	0.5144957554	0.5064260364	0.2790097674
ಪ್ರದರ್ಶನ ಸಂಯೋಜನೆ ಫಾರ್ ಡಿಸಿಗ್ನಾ ರಲ್ಲಿ ಯುನೈಟೆಡ್ ಸ್ಟೇಟ್ಸ್ ದಂಡ	ಪ್ರದರ್ಶನ ಸಂಯೋಜನೆ ಫಾರ್ ಡಿಸಿಗ್ನಾ ರಲ್ಲಿ ಯುನೈಟೆಡ್ ಸ್ಟೇಟ್ಸ್ ದಂಡ	1	0.3333333333	0.6965719461	0.2051282051
ಗಣ್ಯ ಆಫ್ ಮ್ಯಾಕ್ಸಿಮಲ್ ಉತ್ಪಾದನಾ ಬಾಹ್ಯಾಕಾಶ ಗಣ್ಯ ಆಫ್ ಮ್ಯಾಕ್ಸಿಮಲ್	ಗಣ್ಯ ಆಫ್ ಮ್ಯಾಕ್ಸಿಮಲ್ ಉತ್ಪಾದನಾ ಬಾಹ್ಯಾಕಾಶ ಗಣ್ಯ ಆಫ್ ಮ್ಯಾಕ್ಸಿಮಲ್	0	0.6172133998	0.6628050208	0.4411764706
ಕಂಪನಿಯು ಬದಲಿ ಮತ್ತು ರಿಪೇರಿ ಇ ಅದರ ಕಂಪನಿಯ ಅಧಿಕಾರಿಗಳು ಬ	ಕಂಪನಿಯು ಬದಲಿ ಮತ್ತು ರಿಪೇರಿ ಇ ಅದರ ಕಂಪನಿಯ ಅಧಿಕಾರಿಗಳು ಬ	0	0.2020305089	0.4139203429	0.05
ಇತ್ಯರ್ಥವಿರುವ ಕಂಪನಿಗಳು ಹೂ ಒಪ್ಪಂದದ ಅಡಿಯಲ್ಲಿ ಇತ್ಯರ್ಥವಿರುವ	ಇತ್ಯರ್ಥವಿರುವ ಕಂಪನಿಗಳು ಹೂ ಒಪ್ಪಂದದ ಅಡಿಯಲ್ಲಿ ಇತ್ಯರ್ಥವಿರುವ	1	0.7112867592	0.9673586488	0.5555555556
ಎರ್ ಕಮೋಡೋರ್ ಕ್ಲೈಪ್ಸ್ ಅವರು ಭದ್ರತಾ ಕಾರ್ಯಾಚರಣೆಯ ಅಭೂ	ಎರ್ ಕಮೋಡೋರ್ ಕ್ಲೈಪ್ಸ್ ಅವರು ಭದ್ರತಾ ಕಾರ್ಯಾಚರಣೆಯ ಅಭೂ	0	0.4588314677	0.4676598012	0.2962962963
ವಾರ್ ಕಮೋಡೋರ್ ಕ್ಲೈಪ್ಸ್ ಅವರು ಭದ್ರತಾ ಕಾರ್ಯಾಚರಣೆಯ ಅಭೂ	ವಾರ್ ಕಮೋಡೋರ್ ಕ್ಲೈಪ್ಸ್ ಅವರು ಭದ್ರತಾ ಕಾರ್ಯಾಚರಣೆಯ ಅಭೂ	1	0.5262013606	0.5247095823	0.3
ಮೊಸ್ಕಿ ಮತ್ತು ಹಿರಿಯ ಸಹಾಯಕರ ನೆವಾಡಾ ನಲ್ಲಿನ ಎರ್ ಪೋರ್ಟ್	ಮೊಸ್ಕಿ ಮತ್ತು ಹಿರಿಯ ಸಹಾಯಕರ ನೆವಾಡಾ ನಲ್ಲಿನ ಎರ್ ಪೋರ್ಟ್	1	0.5724197752	0.594835043	0.3714285714
ನಿರಾಕರವಾದ ಸ್ವಾತಂತ್ರ್ಯ & ಪೂಜೆ ಬೆಕ್ಕಾಲ್ವೆಲೆಗ್ಗ ನಾನ್ಯಾ ಕಾಂಪೋ	ನಿರಾಕರವಾದ ಸ್ವಾತಂತ್ರ್ಯ & ಪೂಜೆ ಬೆಕ್ಕಾಲ್ವೆಲೆಗ್ಗ ನಾನ್ಯಾ ಕಾಂಪೋ	0	0.3338489304	0.6691991091	0.1785714286

Fig. 4. Dataset with similarity metrics extracted

G. Classification

The reasons for classification are as follows:

- To assign weights for each different similarity metric
- To classify whether the two given sentences are paraphrases or not

To this end, several classifiers that are state-of-the-art were implemented to provide a deep insight into many parameters of the dataset and techniques used. The similarity metrics that are fed into a classifier are the 3 similarity metrics extracted, Cosine Similarity, Cosine Similarity with Bag of Words and Jaccard's Similarity. This is done so as to find their level of significance, also called weights, in deciding whether the sentences are paraphrases or not. The classifiers used are both classification-based models and regression-based models. They include the following:

- Decision Trees
- Random Forest
- Naive Bayes' Classifier
- K-nearest neighbours
- Support Vector Machine

It became increasingly evident that a method was required to make the best predictions, and to remove the generalization

factor of the above mentioned classifiers. To this end, an ensemble model was used so that all the implemented classifiers can be aggregated to overcome the high variance issue. The ensemble model is used in two different ways:

- The above 5 classifiers are trained on the 3 input metrics, and each of these are then aggregated to form an ensemble model.
- The classifier that provides the best result, the Decision Tree, is trained individually for each of the different input metric, and these are then combined to form an ensemble model.

IV. RESULTS AND DISCUSSION

A blackbox approach was taken for viewing the results so that the system can be viewed in it's entire form. The results thus obtained have been interpreted to obtain the final research findings and conclusion.

A. Tokenizer

The input to the tokenizer are the pre-processed Kannada sentences. Each of these sentences are processed by the tokenizer and are converted into tokens. The output is as shown in Fig. 5

```
s1 = "ಅವರು ಶಾಲೆಗೆ ಹೋಗುವುದಿಲ್ಲ"
print(list(tokenize(s1,'kn')))

['_ಅವರು', '_ಶಾಲೆಗೆ', '_ಹೋಗು', 'ುವುದಿಲ್ಲ']
```

Fig. 5. Output of the Tokenizer

B. Stemmer

The input to the stemmer are the Kannada tokens pertaining to each sentence. Each of the sentence tokens are processed and the stemmer rules are applied to remove inflectional words. The output of the stemmer module can be observed in Fig. 6.

```
token = ["ಶಾಲೆಗೆ", "ಹೋಗುವುದಿಲ್ಲ"]
y = []
for i in token:
    flag = 0
    inde = []
    root, inde = kannada_root(i, inde)
    y.append(root)

print(y)

['ಶಾಲೆ', 'ಹೋಗುವು']
```

Fig. 6. Output of the Stemmer

C. Similarity Metrics

The input provided to all of the similarity metric computation modules are the stemmed sentences as array data types in Python. Their similarity metrics are computed to find out whether the sentence pairs are paraphrases or not. The output for each of the similarity metrics are in terms of percentage of syntactic similarity for a sentence pair.

D. Computing Optimum Threshold

For each of the three similarity metrics, it is vital to evaluate their performance. Hence, their accuracy is computed, their values for varying threshold values from 0 to 1 are observed and the best threshold for each of the similarity metrics are retrieved. Once the best possible threshold for the dataset is found, it can be said with high confidence that the actual threshold for a much larger dataset would be in the immediate ballpark of the computed threshold. The experimental results for the same can be observed in Table II. Fig. 7 plots the threshold values against the accuracy achieved for each similarity metric.

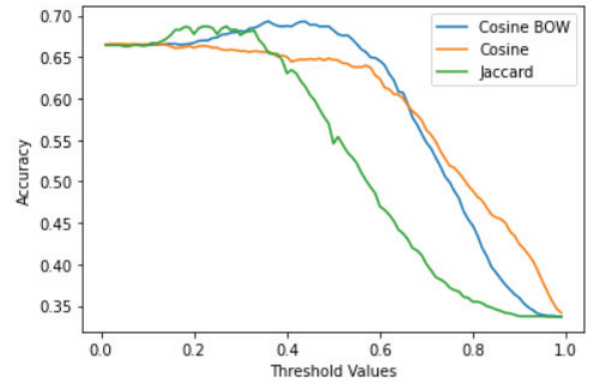


Fig. 7. Threshold vs. Accuracy graph for Jaccard's Similarity

TABLE II
HIGHEST ACCURACY AND CORRESPONDING THRESHOLD VALUE FOR EACH SIMILARITY METRIC

Metric	Jaccard's Similarity	Cosine Similarity	Cosine Similarity with BoW
Highest Accuracy	68.7%	66.6%	69.3%
Threshold	0.22	0.02	0.36

E. Classification

Each learning model's performance was estimated based on its accuracy. Accuracy is a metric of evaluation of learning models that gives the ratio of total number of correct predictions, a combination of true positives and true negatives, to the total number of predictions by learning model. It can be seen from Table III that Decision Tree performed the best. This is attributed to the dataset being non-linear and hence, the

model made better predictions based on the three metrics that were given as inputs. Moreover, the other classifiers perform worse for various reasons, including linearity problem, no correlation between the different inputs and the necessity for large datasets.

TABLE III
ACCURACY ACHIEVED BY THE SEVERAL CLASSIFIERS

Classifier model	Accuracy (%)
SVM	65.58
KNN	62.39
Random forest	65.75
Decision tree	86.6
Naive-Bayes' Classifier	69

V. CONCLUSION

An approach to solving the problem of detecting paraphrases in the Kannada language is solved in this paper. Extensive literature review, building stemmer rules from the ground up and training several models, all were extremely relevant and challenging tasks that were executed to fruition. The high accuracy of Decision Tree model can be attributed to the non-linearity of the data points present in the dataset. The novelty of the project and such an implementation model shows that there is much to be researched regarding the different feature extraction methods and deep learning models that can be implemented to increase the span of research for low resourced languages.

VI. FUTURE SCOPE

A Paraphrase detection tool can be implemented in machine translation, question answer, plagiarism detection, information retrieval and, validate paraphrase generation. Paraphrase detection can improve question-answering by a significant amount as it can recognize the reoccurring or paraphrased questions and answers. Paraphrase detection can also improve the performance of information retrieval by paraphrasing the queries being sent. The tool can also be extended to an automated plagiarism detection and a paraphrase plagiarism detection.

REFERENCES

- [1] Chris Quirk, Chris Brockett, and William Dolan. Monolingual machine translation for paraphrase generation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 142–149, Barcelona, Spain, July 2004. Association for Computational Linguistics.
- [2] Bill Dolan, Chris Quirk, and Chris Brockett. Unsupervised construction of large paraphrase corpora: Exploiting massively parallel news sources. In *COLING 2004: Proceedings of the 20th International Conference on Computational Linguistics*, pages 350–356, Geneva, Switzerland, aug 23–aug 27 2004. COLING.
- [3] Ashwini I Gadag and B M Sagar. N-gram based paraphrase generator from large text document. In *2016 International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS)*, pages 91–94, 2016.
- [4] Amba Kulkarni, Soma Paul, Malhar Kulkarni, Anil Kumar, and Nitesh Surtani. Semantic processing of compounds in indian languages. pages 1489–1502, 12 2012.
- [5] Kamal Sarkar. Ks_ju@dpil-fire2016: Detecting paraphrases in indian languages using multinomial logistic regression model. *CoRR*, abs/1612.08171, 2016.
- [6] R.J. Prathibha and M.C. Padma. Design of rule based lemmatizer for kannada inflectional words. In *2015 International Conference on Emerging Research in Electronics, Computer Science and Technology (ICERECT)*, pages 264–269, 2015.
- [7] Yashaswini Hegde, Shubha Kadambe, and Prashantha Naduthota. Suffix stripping algorithm for kannada information retrieval. In *2013 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, pages 527–533, 2013.
- [8] S. Parameswarappa, V. N. Narayana, and G. N. Bharathi. A novel approach to build kannada web corpus. In *2012 International Conference on Computer Communication and Informatics*, pages 1–6, 2012.
- [9] Danish Mohammed Ebadulla, Rahul Raman, Hridhay Kiran Shetty, and Mamatha H.r. A comparative study on language models for the Kannada language. In *Proceedings of the Fourth International Conference on Natural Language and Speech Processing (ICNLSP 2021)*, pages 280–284, Trento, Italy, 12–13 November 2021. Association for Computational Linguistics.
- [10] Shruti Srivastava and Sharvari Govilkar. A survey on paraphrase detection techniques for indian regional languages. *International Journal of Computer Applications*, 163:42–47, 04 2017.